



Deployments and Services for AI Applications

A practical guide for DevOps and ML engineers deploying AI applications in Kubernetes environments

Why Deployments and Services?



The Pod Problem

Pods are **ephemeral** by design and can fail at any time, making them unreliable for production AI workloads



Deployment Solution

Deployments manage pod lifecycles, ensuring your AI applications maintain the desired state despite failures



Service Access

Services provide stable network endpoints, making your AI APIs consistently accessible

Together, Deployments and Services form the foundation for reliable, scalable AI applications in Kubernetes.

What Is a Deployment?

A Deployment is a Kubernetes object that:

- **Manages pod lifecycles** automatically
- Ensures the desired number of replicas are running
- Handles [updates & rollbacks](#) with zero downtime
- Abstracts away pod failures through self-healing
- Provides declarative updates for your AI applications

Deployments are essential for scaling stateless AI APIs and managing ML inference jobs in production.



Deployment Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ai-api
  template:
    metadata:
      labels:
        app: ai-api
    spec:
      containers:
        - name: api
          image: ai-app:latest
          ports:
            - containerPort: 8000
```

What This Does:

- Creates **3 identical replicas** of the AI API pod
- Self-heals if any pod crashes or becomes unhealthy
- Uses labels to track and manage all related pods
- Exposes port 8000 for the application



What Is a Service?



A Service is a Kubernetes abstraction that:

- Provides a **stable network endpoint** for accessing pods
- Load-balances traffic across all available replicas
- Decouples frontend clients from backend pod implementations
- Enables discovery through DNS within the cluster
- Comes in multiple types for different access patterns

Services are critical for exposing AI inference APIs to users and connecting microservices within your ML pipeline.

Service Example

```
apiVersion: v1
kind: Service
metadata:
  name: ai-service
spec:
  selector:
    app: ai-api
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
  type: LoadBalancer
```

This example creates a LoadBalancer service that:

- Maps external port 80 to container port 8000
- Automatically load balances requests across all replicas
- Provides a stable external IP address (via cloud provider)



- ❶ The **selector** field connects the Service to all Pods with the matching label (`app: ai-api`), regardless of which nodes they're running on.

Types of Services

1

ClusterIP

Default type that exposes the Service on an internal cluster IP

- Only accessible *within* the cluster
- Ideal for internal AI microservices

2

NodePort

Exposes the Service on each Node's IP at a static port

- Accessible externally via `NodeIP:NodePort`
- Good for development and testing

3

LoadBalancer

Exposes the Service externally using a cloud provider's load balancer

- Creates an external IP that routes to the Service
- **Recommended for production AI APIs**

4

Headless

Special service (ClusterIP: None) that returns individual Pod IPs

- Direct Pod-to-Pod communication
- Useful for stateful workloads

Deployments for AI Workloads

Training Jobs

Often managed via specialized custom controllers or operators like KubeFlow, rather than standard Deployments

Inference APIs

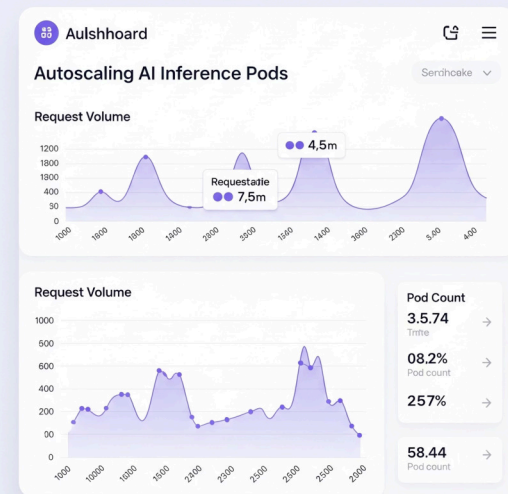
Run as Deployments to handle variable traffic loads and ensure high availability for model serving

Auto-scaling

Scale replicas up/down based on CPU, memory, or custom metrics like prediction request volume

Model Updates

Use rolling updates to gradually replace pods running older model versions with zero downtime



📄 **Canary deployments** let you test new AI models with a small percentage of traffic before full rollout.

Services for AI Workloads



API Exposure

Expose model servers to users via stable, load-balanced endpoints that persist even as pods come and go



Data Access

Connect inference pods with databases, vector stores, and feature stores using internal services

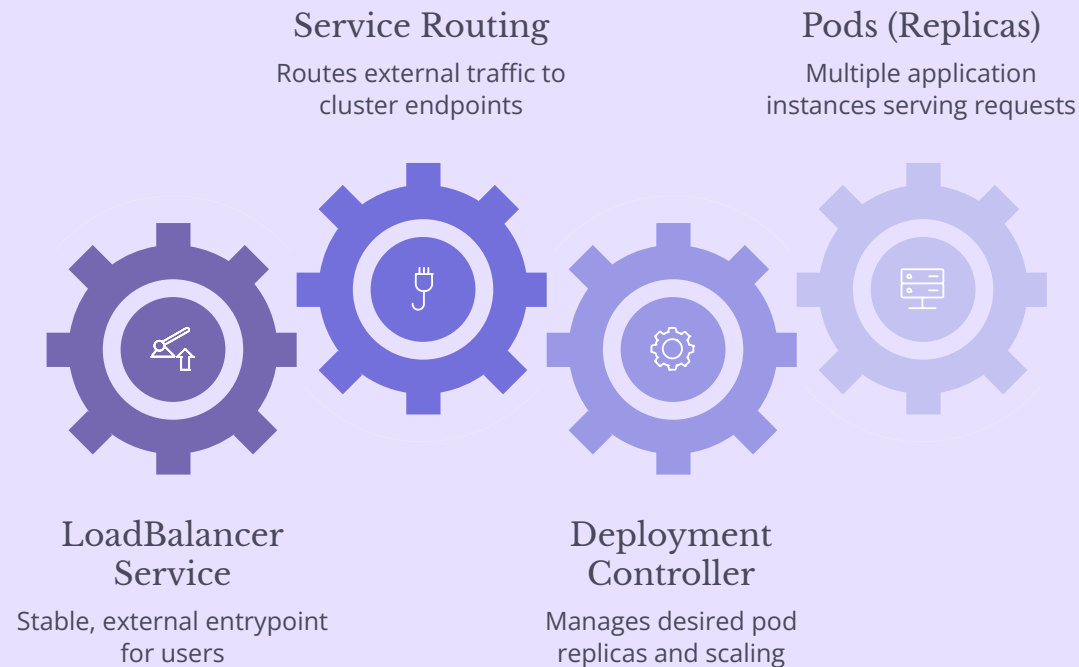


Security

Secure AI services with TLS certificates, authentication, and ingress controllers for traffic management



Deployments + Services Together



This combination creates a resilient, user-facing AI application architecture:

- Deployment ensures the **right number of pods** are always running
- Service provides a **stable endpoint** for accessing those pods
- Together they enable production-grade AI inference APIs with high availability
- Example: GPT inference API with multiple replicas and load balancing

Key Takeaways

Deployments

- Manage pod scaling, updates, and self-healing
- Ensure desired state despite infrastructure failures
- Enable zero-downtime updates for AI model versions

Services

- Provide stable endpoints to access your AI applications
- Load balance requests across available pods
- Enable external access through various service types

Next Steps

Implement health checks for AI pods

Set up resource limits and auto-scaling

Configure proper service monitoring

Explore Ingress for advanced routing

These fundamentals are the **foundation for production-grade AI infrastructure** on Kubernetes.